

NEC Vector Annealing PoC version Manual 2nd Edition

Precautions When Exporting

This product (including the software) may correspond to a regulated good (or service) as defined by the Foreign Exchange and Foreign Trade Act.

In that case, an export license from the Japanese government is required when exporting the product outside of Japan.

If related documentation, etc. is required for the export license application procedure, please consult with the dealer from which the product was purchased or a nearby NEC sales office.

Preface

- ◆ This document explains how to install, configure the environment for, and use NEC Vector Annealing (VA) on the SX-Aurora TSUBASA system.

- ◆ Trademarks and copyrights
 - Linux is a trademark of Linus Torvalds in the United States and other countries.
 - Red Hat and Red Hat Enterprise Linux are trademarks registered by Red Hat, Inc. in the United States and other countries.
 - Other listed company names and product names are registered trademarks or trademarks of their respective owners.

Terminology Definitions and Abbreviations

Terminology and abbreviation	Explanation
Vector Engine (VE)	The VE is the core of SX-Aurora TSUBASA which performs the vector operations. It is a PCI Express card which is installed and used on an x86 server.
Vector Host (VH)	Refers to the server which maintains the vector engine, in other words, the host computer.
Annealing	In this document, refers to simulated annealing, which is a method to probabilistically seeks a near-optimal solution by gradually decreasing the "temperature."

Terminology	Data type	Description example	Definition
Spin name	Character string	'x[0][0]', 'x[0][1]', ...	Name of each spin (must be unique) <i>*Note that when the spin names are specified with a mix of the character string type and byte string type, such as 'x' and b'x', they are treated as different spins.</i>
Spin state	Integer	0 or 1	Each spin state is described as 0/1
Spin	[Spin name, spin state]	['x[0][0]', 0], ...	Expressed as a combination of the spin name and spin state
Convergence energy	Real number*		Total energy within the model
Constraint state	Boolean variable		Constraint satisfied = True, not satisfied = False
Operation time	Real number*		Annealing calculation time

*: VA processes using single precision floating point arithmetic.

Therefore, real number parameters specifiable in the Python API must be specified in a range that can be expressed as a single precision floating point (1.175494×10^{-38} to 3.402823×10^{38}).

If values outside of the range above are specified, they are treated as infinity and result in an error.

Operating Requirements and Precautions

◆ Operating requirements

- Linux verified to run on the SX-Aurora TSUBASA
 - RHEL 7.9
 - CentOS 7.9
- SX-Aurora TSUBASA related software
 - SDK Runtime (free)
- VE card requirements
 - VEOS 2.10.1 or higher
 - 48 GB of memory or more (essential for full coupling of 100,000 bits)
 - *Full coupling up to 70,000 bits is supported with 24 GB

◆ Precautions

- In the case of using on premise type, the run time is possible to be prolonged when one VE card is shared between multiple users and the annealing processes conflict with one another. Therefore, it is recommended that you implement a job queue control to avoid inserting the next process until the previous process ends. The VA package includes “job management server” to provide the queue control functions.

Index

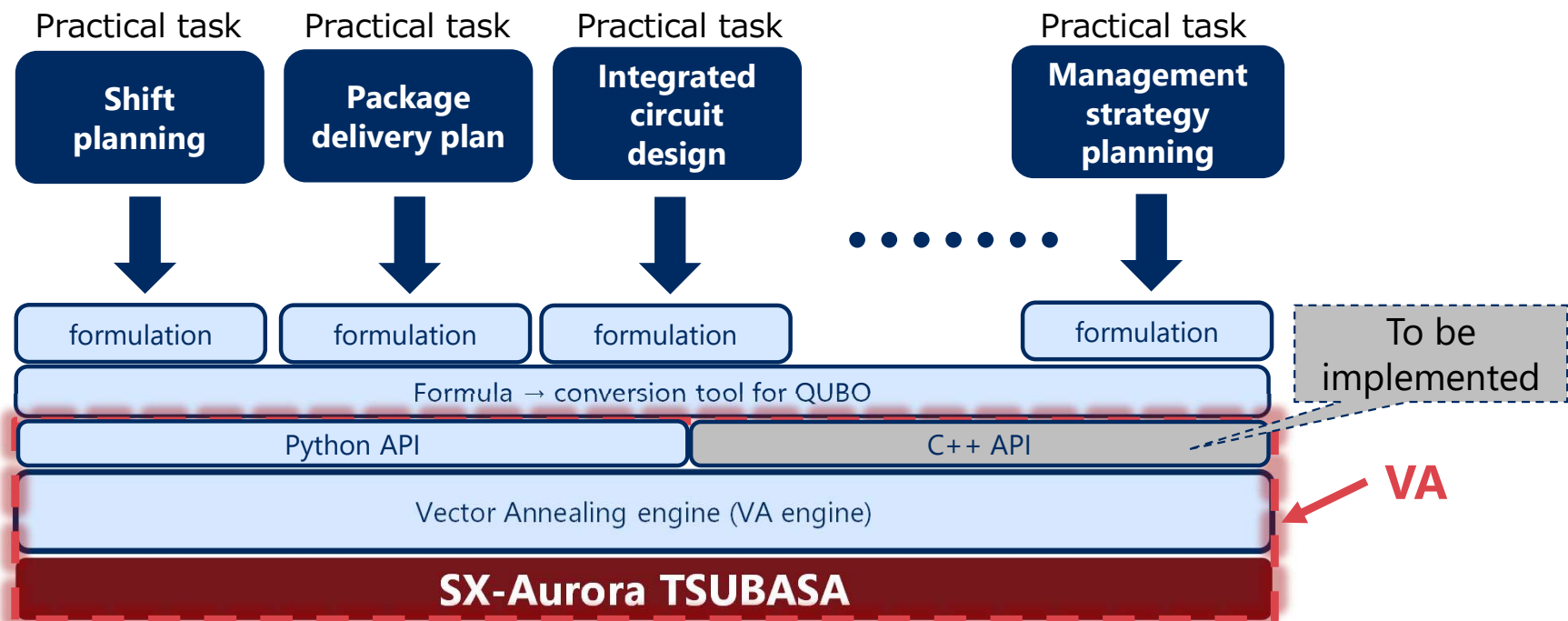
1. What is NEC Vector Annealing (VA)?
2. VA's search algorithm taking account of constraints
3. Preparation Before Using VA
4. How to use VA module
5. Parameter Supplement
6. How to use Flip Option
7. How to specify the initial spin state
8. How to uninstall VA

What is NEC Vector Annealing (VA)?

The machine implemented in SX-Aurora TSUBASA for solving formalized combinatorial optimization problems by simulated annealing

Specification

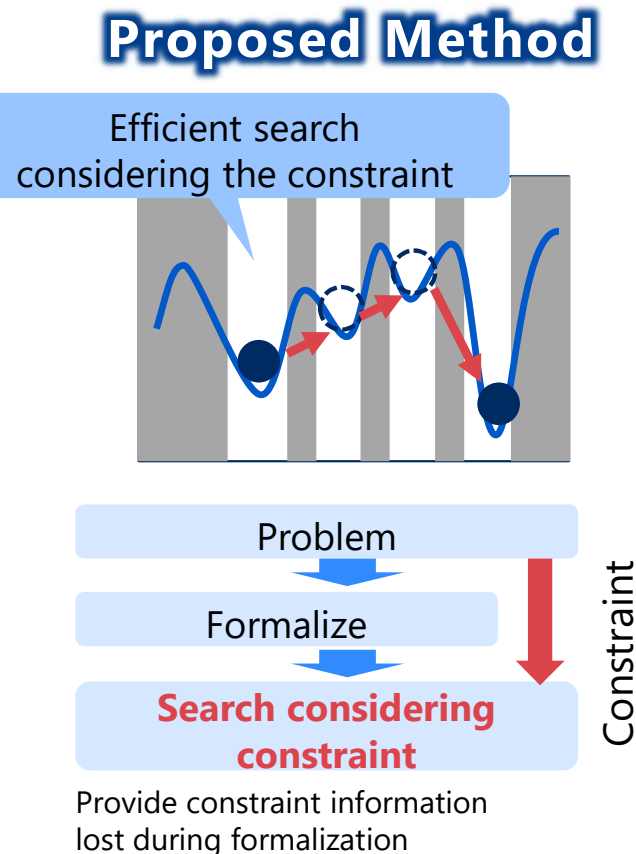
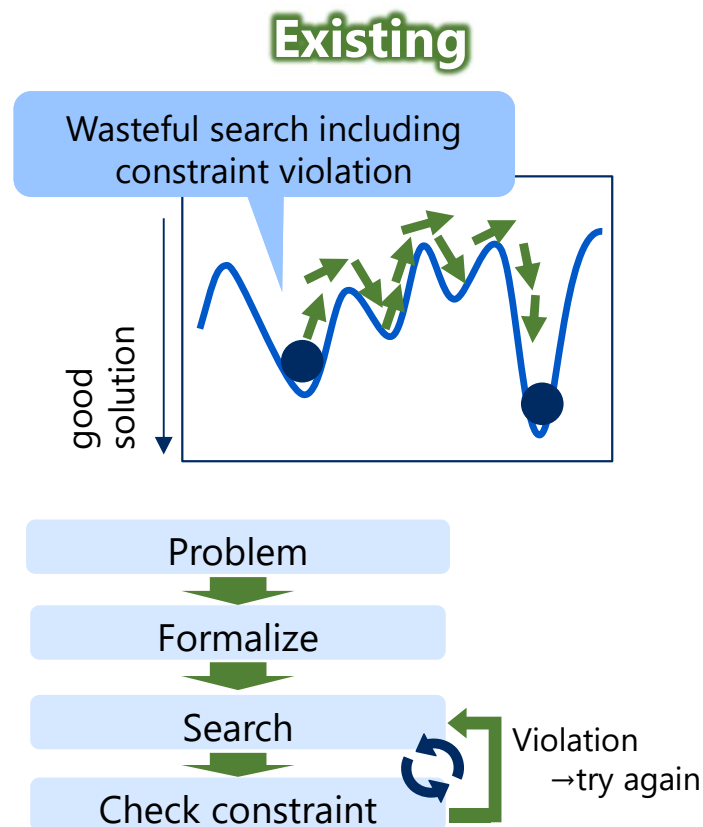
- Input format: QUBO
- Binary data size: up to 100 thousand bit
- Combination type: full connection and 32bit resolution
- Output format: value of binary variables (0/1)



VA's search algorithm taking account of constraints

We prepare for the **flip option** that is able to reduce the amount of calculation by searching only in the solution space satisfied with constraints.

- E.g. In case of the constraint that only one spin(0/1) from x_1 to x_{10} should be one, $x_1 \sim x_{10}$ will be flipped in a set to be satisfied with above the constraint when the spin is flipped during the search.



Preparation Before Using VA (1/3)

It is necessary that Python3.8 can be used, and numpy (1.22.3 or higher) and pyqubo* (1.0.13 or higher) can be used.

1. Environment settings

- Run the following commands to configure the environment variables. **(Required during login)**
These commands may be omitted if the settings are described in the shell configuration file that is run during login.

```
$ source /opt/nec/ve/nlc/2.3.0/bin/nlcvars.sh  
$ export PATH=${PATH}:/opt/nec/ve/bin
```

Adjust to the version of your environment

2. Place the VectorAnnealing-VApoc_0201.zip file that was obtained into any folder using root privileges and extract the tool into /opt/va/VApoc_0201.

```
$ cd /opt  
$ tar zxf File path/VectorAnnealing-VApoc_0201.zip
```

3. Add the following environment variables to the user's .bashrc.

```
export PYTHONPATH=/opt/va/VApoc_0201/libexec/VectorAnnealing/python:${PYTHONPATH}
```

* : Pyqubo definition

A domain specific language (DSL) that converts a formulated formula to the QUBO format when solving a combinatorial optimization problem on an annealing machine. It can be used in the same way as other Python libraries.

<https://pyqubo.readthedocs.io/en/latest/index.html>

<https://github.com/recruit-communications/pyqubo>

Preparation Before Using VA (2/3)

4. Run the sample program (NP.py) under /opt/va/VApoc_0201/samples/NP/python to verify that it operates normally.

/tmp is any folder to which you have write access

```
$ cd /tmp  
$ python3.8 /opt/va/VApoc_0201/samples/NP/python/NP.py
```

```
[HINTS]  
8 . . | . . . | . . .  
...Omitted
```

```
[SOLUTION]  
8 1 2 | 7 5 3 | 6 4 9  
9 4 3 | 6 8 2 | 1 7 5  
6 7 5 | 4 9 1 | 2 8 3  
-----+-----+-----  
1 5 4 | 2 3 7 | 8 9 6  
3 6 9 | 8 4 5 | 7 2 1  
2 8 7 | 1 6 9 | 5 3 4  
-----+-----+-----  
5 2 1 | 9 7 4 | 3 6 8  
4 3 8 | 5 2 6 | 9 1 7  
7 9 6 | 3 1 8 | 4 5 2
```

Operation is normal if a SOLUTION such as the one on the left is output
***The following type of WARNING may sometimes appear, and it means that it could not find a solution, so run it again.**

```
2022-05-30 10:39:45.120 [Sampler] WARNING : va_qubo.exe  
output result with broken constraint.  
energy : 4.000 INVALID
```

5. This completes the preparation.

Preparation Before Using VA (3/3)

Reference: other sample code (traveling salesman problem)

/opt/va/VApoc_0201/samples/TSP/python/TspLib.py

- Download and extract [ALL tsp.tar.gz](#) from the following site and store a problem file (ex: eil51.tsp) in any folder to which you have write access.
 - *In the list of available files, only the "EDGE_WEIGHT_TYPE: **EUC_2D**" problems are supported (ex: eil51.tsp)
<http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/>
 - *Large-scale problem files with **100 cities or more** will cause the QUBO to increase in size and Pyqubo may be unable to generate it, so please avoid using such files.
- The file can be executed as shown in the following execution example.

```
$ cd /tmp
$ python3.8 /opt/va/VApoc_0201/samples/TSP/python/TspLib.py eil51.tsp
Read eil51.tsp
Energy, Length
89.000, 437.000
-----
[Result]
Turn[] = [0, 21, 1, 28, 19, 35, 34, 2, 27, 30, 25, 7, 47, 22, 6, 42, 23, 13, 5, 26, 50, 45,
11, 46, 3, 17, 24, 12, 40, 18, 39, 41, 43, 16, 36, 14, 44, 32, 9, 38, 29, 33, 20, 15, 49, 8,
48, 4, 37, 10, 31]
length = 437
```

Any folder to which you have write access

Downloaded TSPLIB file

Result
distance

An array that represents the order of city
tours. The city number is show as 0 origin.

How to use VA module (1/2)

Create QUBO and execute annealing

```
...  
  
# make a qubo  
param_C = max_len * 0.25  
qubo, offset = model.to_qubo(feed_dict={'num_a': param_C})  
  
# -----  
# Import Vector Annealing module  
import VectorAnnealing  
  
# Make a Vector Annealing Model  
va_model = VectorAnnealing.model(qubo, offset)  
  
# Make a sampler  
sa = VectorAnnealing.sampler()  
  
# Execute Simulation(Number of execution times is 5)  
result = sa.sample(va_model, num_reads=5)
```

Annealing result is returned in the following array of result class for each annealing.

```
-----  
class result:  
    def __init__(self, spin=None, energy=None, time=None, constraint=None, memory_usage=None):  
        self.spin = spin # Spin state of QUBO. Ex) spin = { 'x':0, 'y':0, 'z':1 }  
        self.energy = energy # Total energy of QUBO.  
        self.time = time # Simulation time of this result.  
        self.constraint = constraint # Satisfy constraint if constraint is 'True', Broken constraint if constraint is 'False'.  
        self.memory_usage = memory_usage # Memory usage of simulation.  
    -----
```

How to use VA module (2/2)

Make a QUBO by using pyqubo

- Use the sample code of Traveling salesman problem

```
# import pyqubo
from pyqubo import Array, Placeholder

# Define qubo with two dimentions : x[point_num][point_num]
x = Array.create('x', shape=(point_num, point_num), vartype='BINARY')

# Write a formula
param_a = Placeholder('num_a')
Hd = sum(sum((dlength[j][k] - d_min[j]) * x[i, j] * x[(i+1)%point_num, k] for i in range(point_num)) for (j,k) in pairs)
Ha = param_a * sum((sum(x[i, j] for i in range(point_num)) - 1)**2 for j in range(point_num))
Hb = param_a * sum((sum(x[i, j] for j in range(point_num)) - 1)**2 for i in range(point_num))
H = Hd + Ha + Hb

# Make a model
model = H.compile()

# Make a qubo
param_C = max_len * 0.25
qubo, offset = model.to_qubo(feed_dict={'num_a': param_C})
```

Parameter Supplement

Sampler class parameter specification

- The `sa.sample()` function which runs the annealing supports the following parameters in addition to `num_reads`.

Parameter name	type	default	Description
<code>num_reads</code>	Integer type	1	Number of samplings
<code>num_results</code>	Integer type	None	Number of results. Returns one optimal solution when None or 1 is specified. Returns all annealing results when the same value as with <code>num_reads</code> is specified.
<code>num_sweeps</code>	Integer type	500	Number of annealing sweeps
<code>beta_range</code>	[real number type*, real number type*, integer type]	[10,100,200]	Specify the start and end values of the β parameter which is the inverse of the temperature in [start,end,nsteps]. nsteps is the number of divisions from start to end and can be omitted.
<code>init_spin</code>	Spin array	None	Initial VA spin state specification (For more details, see the "Initial VA spin state specification" section below.)
<code>vector_mode</code>	str type	accuracy	Annealing is performed in speed priority mode when set to SPEED. Annealing is performed in accuracy priority mode when set to ACCURACY.

*: VA processes using single precision floating point arithmetic.

Therefore, real number parameters specifiable in the Python API must be specified in a range that can be expressed as a single precision floating point (1.175494×10^{-38} to 3.402823×10^{38}).

If values outside of the range above are specified, they are treated as infinity and result in an error.

How to use Flip Option (1/8)

How to use Flip option

- By specifying Flip option, you can efficiently obtain the results in a simulation.

***These options must be included in Hamiltonian's formulation.**

- Sample code: Traveling salesman problem

```
# create one hot constraint rule.
onehot = [0] * (2 * point_num)
for i in range(point_num):
    onehot1 = [0] * point_num
    onehot2 = [0] * point_num
    for j in range(point_num):
        onehot1[j] = 'x[%d][%d]' % (i, j)
        onehot2[j] = 'x[%d][%d]' % (j, i)
    onehot[2*i] = onehot1
    onehot[2*i+1] = onehot2
```

Define constraints on Onehot

Specify the constraint so that only one city becomes 1 at order i and only one order becomes 1 at city j.

```
# create fixed spin constraint rule.
fixed = []
for i in range(point_num):
    for j in range(point_num):
        if (j == 0) and (i == 0):
            fixed.append(['x[0][0]', 1])
        elif (j != 0) and (i == 0):
            fixed.append(['x[0][%d]' % j, 0])
        elif (j == 0) and (i != 0):
            fixed.append(['x[%d][0]' % i, 0])
```

Define constraints on Fixed spin

Specify $x[0][0] = 1$ and $x[0][*] = x[*][0] = 0$ in order to start from the city at $j = 0$

Specify the constraint of Flip options when creating model of Vector Annealing.

```
va_model = VectorAnnealing.model(qubo, offset, onehot=onehot, fixed=fixed)
```

How to use Flip Option (2/8)

How to specify Flip option

- One hot constraint
 - The constraint that one of the spin states is "1" in the specified group of spin

A group of One hot constraints

```
one_hot_list = [  
    ['x[0][0]', 'x[0][1]', 'x[0][2]', 'x[0][3]', 'x[0][4]'],  
    ['x[1][0]', 'x[1][1]', 'x[1][2]', 'x[1][3]', 'x[1][4]'],  
    ['x[2][0]', 'x[2][1]', 'x[2][2]', 'x[2][3]', 'x[2][4]'],  
]  
  
va_model = VectorAnnealing.model(qubo, offset, onehot=one_hot_list)
```

Define multiple one hot constraints

Specify the defined one hot condition in the variable of onehot

How to use Flip Option (3/8)

How to specify Flip option

- Fixed spin constraint

**Only this option does not need to be included in Hamiltonian's formulation.*

- The constraint that specifies the state of spin to the specified value (0/1)

**Specify set of the name of spin
and the state of spin(0/1)**

```
fixed_spin_list = [  
    [ 'x[0][0]', 1 ],  
    [ 'x[0][1]', 0 ],  
    [ 'x[0][2]', 0 ],  
    [ 'x[0][3]', 0 ]  
]
```

Specify the status of multiple spin

```
va_model = VectorAnnealing.model(qubo, offset, fixed=fixed_spin_list)
```

How to use Flip Option (4/8)

How to specify Flip option

- And zero constraint
 - The constraint that at least one of the spin states is "0" in the specified group of spin

A Group of And zero constraint

```
and_zero_list = [  
    ['x[0][0]', 'x[0][1]', 'x[0][2]', 'x[0][3]', 'x[0][4]'],  
    ['x[1][0]', 'x[1][1]', 'x[1][2]', 'x[1][3]', 'x[1][4]'],  
    ['x[2][0]', 'x[2][1]', 'x[2][2]', 'x[2][3]', 'x[2][4]'],  
]
```

Define multiple and zero constraint

```
va_model = VectorAnnealing.model(qubo, offset, andzero=and_zero_list)
```

Specify the defined and zero condition in the variable of andzero

How to use Flip Option (5/8)

How to specify Flip option

- Or one constraint
 - The constraint that at least one of the spin states is "1" in the specified group of spin

```
or_one_list = [  
    ['x[0][0]', 'x[0][1]', 'x[0][2]', 'x[0][3]', 'x[0][4]'],  
    ['x[1][0]', 'x[1][1]', 'x[1][2]', 'x[1][3]', 'x[1][4]'],  
    ['x[2][0]', 'x[2][1]', 'x[2][2]', 'x[2][3]', 'x[2][4]'],  
]
```

A Group of Or one constraint

Define multiple or one constraint

```
va_model = VectorAnnealing.model(qubo, offset, orone=or_one_list)
```

Specify the defined or one condition in the variable of orone

How to use Flip Option (6/8)

How to specify Flip option

- Cubic supplement constraint

- The constraint that $\text{spin}(x_1, x_2, y_1)$ always has a value that satisfies expression $y_1 = x_1 x_2$

Describe the constraints of cubic supplement in the order of y_1, x_1, x_2

```
spl_list = [  
    [ 'y[0]', 'x[0][0]', 'x[0][1]' ],  
    [ 'y[1]', 'x[1][0]', 'x[1][1]' ],  
    [ 'y[2]', 'x[2][0]', 'x[2][1]' ],  
]  
  
va_model = VectorAnnealing.model(qubo, offset, spl=spl_list)
```

Define multiple cubic supplement constraint

Specify the defined cubic supplement condition in the variable of spl

***Need the following Hamiltonian that satisfied expression $y_1 = x_1 x_2$**

Hamiltonian formulation sample)

```
Hp1 = x[0,0]*x[0,1]-2*x[0,0]*y[0]-2*x[0,1]*y[0]+3*y[0]  
Hp2 = x[1,0]*x[1,1]-2*x[1,0]*y[1]-2*x[1,1]*y[1]+3*y[1]  
Hp3 = x[2,0]*x[2,1]-2*x[2,0]*y[2]-2*x[2,1]*y[2]+3*y[2]
```

How to use Flip Option (7/8)

How to specify Flip option

- Max one count constraint

- The constraint that the number of spins having the "1" state in the specified group is equal to or less than the specified number.

Specify the maximum number of spins that will be in state "1"

Describe spin group of constraint of Max one count

```
max_one_list = [  
    [2, ['x[0][0]', 'x[0][1]', 'x[0][2]', 'x[0][3]', 'x[0][4]'] ],  
    [1, ['x[1][0]', 'x[1][1]', 'x[1][2]', 'x[1][3]', 'x[1][4]'] ],  
    [1, ['x[2][0]', 'x[2][1]', 'x[2][2]', 'x[2][3]', 'x[2][4]'] ],  
]
```

Define the Max one count constraint

Define multiple max one count constraint

```
va_model = VectorAnnealing.model(qubo, offset, maxone=max_one_list)
```

Specify the defined max one count condition in the variable of maxone

How to use Flip Option (8/8)

How to specify Flip option

• Min max one constraint

- The constraint that the number of spins having the "1" state in the specified group is equal to the specified range.
 - The range parameter is defined as [MIN, MAX, [spin0, spin1, spin2,]]
 - » In the case of MIN = < MAX
MIN = < the number of spins having the "1" state = < MAX
 - » In the case of MIN > MAX
the number of spins having the "1" state = < MAX or MIN = < the number of spins having the "1" state

```
minmax_one_list = [
```

**Specify MIN=1
(Minimum threshold)**

**Describe spin group of
constraint of Min max one**

```
[ 1, 3, [ 'x[0][0]', 'x[0][1]', 'x[0][2]', 'x[0][3]', 'x[0][4]' ] ],
```

**Specify MAX=3
(Maximum threshold)**

Define the Min max one constraint

```
[ 3, 1, [ 'x[1][0]', 'x[1][1]', 'x[1][2]', 'x[1][3]', 'x[1][4]' ] ],
```

**the number of spins having the "1" state = < MAX(=1) or
MIN(=3) = < the number of spins having the "1" state**

```
[ 2, 3, [ 'x[2][0]', 'x[2][1]', 'x[2][2]', 'x[2][3]', 'x[2][4]' ] ],
```

```
]
```

```
va_model = VectorAnnealing.model(qubo, offset, minmaxone=minmax_one_list)
```

Define multiple
min max one constraint

How to specify the initial spin state

How to specify the initial spin state

- You can specify the spin state when annealing starts in the following two ways
 - Specify “Fixed spin” constraint for the model
 - Because “Fixed spin” constraint is the constrained condition that specifies the spin being a fixation, it is possible to apply the state of the specified spin to the initial spin state.
 - Specify the initial spin state at the call of the annealing
- Action
 - specify the initial spin state to the initial value when annealing starts
 - If you specify a different value for the same spin, set the value with the following priority
 1. The condition specified “Fixed spin” constraint for the model
 2. The condition specified the initial spin state at the call of the annealing

Sample code:

```
spin_state = {  
    'x[0][0]':0,  
    'x[1][1]':1,  
    'x[2][3]':1,  
    ...  
}
```

Initial spin state is specified by a combination of spin name and spin state (0/1). Spins that are not included are treated as no initial value, and the initial value is determined by a random spin state at the start of the annealing.

Specify the dictionary type data which stores the initial value of the spin for `init_spin`.

```
sa = VectorAnnealing.sampler()  
result = sa.sample(va_model, num_reads=5, init_spin=spin_state)
```

How to uninstall VA

How to uninstall VA

- 1. Delete the following environment variables set in "Preparation Before Using VA".

```
export PYTHONPATH=/opt/va/VApoc_0201/libexec/VectorAnnealing/python:${PYTHONPATH}
```

- 2. Delete the /opt/va/VApoc_0201 folder using root privileges .

```
$ cd /opt/va  
$ rm -rf VApoc_0201
```

How to uninstall Parameter_estimator

If you have also installed the bundled Parameter_estimator, please uninstall it at the same time.

- Uninstall with the Pip3.8 command.

```
$ pip3.8 uninstall parameter-estimator  
Uninstalling parameter-estimator-1.0.0:  
Would remove:  
  /home/.local/lib/python3.8/site-packages/parameter_estimator-1.0.0.dist-info/*  
  /home/.local/lib/python3.8/site-packages/parameter_estimator/*  
Proceed (y/n)? y  
Successfully uninstalled parameter-estimator-1.0.0
```

Issuance history

◆ Issuance history list

September	2022	1st Edition
November	2022	2nd Edition

◆ Details of additions and changes

1st Edition	New
2nd Edition	Modified the tool name and folder name according to the tool version change

NEC Vector Annealing PoC version Manual

November 2022, 2nd Edition

NEC Corporation
7-1, Shiba 5-chome, Minato-ku, Tokyo
TEL (03) 3454-1111 (Main)

© NEC Corporation 2022

This document may not be duplicated or modified without the permission of NEC Corporation.
The contents of this document are subject to change without notice in the future.

\Orchestrating a brighter world

NEC